

Quicksilver: A Component-Based Environment for Agent-Based Computer Models and Simulations

28. June 2000

Jan Burse,
Swiss Federal Institute of Environmental
Science and Engineering
Überlandstr. 113, 8006 Dübendorf, Switzerland
burse@eawag.ch, www.eawag.ch

Content

Introduction.....	1
The Component Based Approach.....	2
Agent-Based Simulation Components	4
Current Further Development	5
Conclusions	7
References	8

Introduction

The Quicksilver modeling and simulation environment is an ongoing development of the integrated assessment group at EAWAG. The project started two years ago and the environment has already found various applications in the area of sustainable development (Pahl 2000a), water management (Tillman 2000, Hare 2000) and market research (Kottonau 2000). The computer models and simulations that result from the environment can be optionally packed as Java applets and embedded in hypertext information systems. The environment already helped to build such enhanced information systems which were then used in participatory processes such as focus groups (Schlumpf 2000, Pahl 2000b) and personal calculators (Schönborn 1999).

The Quicksilver environment supports a very broad notion of computer model. With the environment it is possible to implement computer models that are based on partial differential equations (Luenberger 1979). But computer models that are purely of this type are rather the exception for this environment. A great deal of effort was put into the support for agent-based computer models. These computer models allow for a rich but still rigorous representation (Chattoe 1996). The notion of an agent-based computer model can be easily combined with partial differential equations and subsequently refined to support the notion of hierarchical structures (Troitzsch 1996), deliberative agents (Doran 1996) and uncertainty analysis (Varis 1994). The development of support for these additional concepts is still an ongoing process.

When we look at the type of computer models we want to build then one has to ask whether it is still possible to build and manage such a richness. In particular one can ask the question by means of economic principles. This means to look for the value of information of the resulting model and to discount the effort to build and manage the model (Moss 2000). This analysis would render many models intractable. To attack this problem we introduce the component-based approach to computer modeling and simulation. This approach has the promise of allowing the evolution and composition of reusable knowledge assets. We have already looked at the component-based approach from an abstract methodological point of view in (Burse 1998). In this paper we will first take a brief look at the economic aspect of the approach. We will then show how the approach is applied. Further more

we will discuss the particular components that have evolved to support our current notion of computer model. At the end we will draw some conclusions.

The Component Based Approach

Agent-based computer models and simulations form a special area of the application of computer systems. Quicksilver is an attempt of an investigation into this area in the light of modern component-based software development. Software development has many years suffered from the lack of a component-based engineering approach. On the other hand this approach is available in many other engineering disciplines. Often for example production processes draw on components that were produced elsewhere. Only recently has the component-based engineering approach found its way into the general software development practice. The transition is still going on and one of the enabling technologies is object orientation. By means of object orientation components can be introduced that are not simply shareable libraries but rather objects that can be extended.

The component-based approach to application development is based on some ontological commitments, i.e. ways we perceive the reality. One ontological commitment is that we assume that an organization is or will deploy a number of application. This is not so debatable as most bigger organizations can be perceived as performing many different functions internal and over their borders. The next ontological commitment is now to view one of the many application inside an organization as a collection of software components. Thus an application is viewed has having a part-whole structure, where the whole is the application itself and the parts are the software components. Often software development methods and tools are mainly based on this view. The task of software development is then defined as a simple decomposition/composition problem, namely as finding the parts that make up the desired whole.

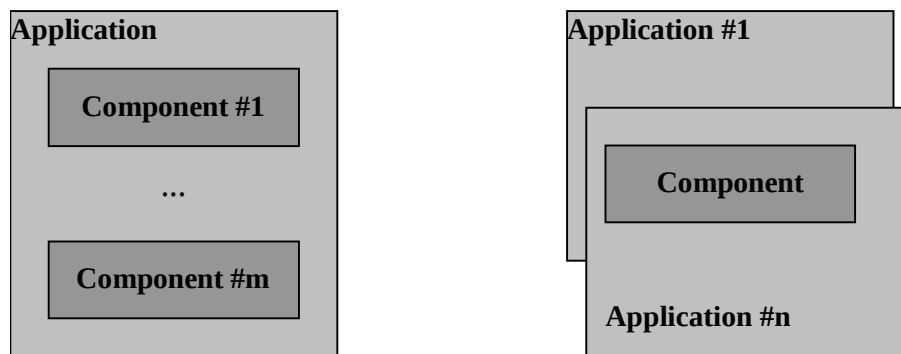


Figure 1: The ontological commitments of the component-based approach

If we view an application as a simple part-whole structure we neglect our initial ontological commitment, namely that we assumed many different applications. The question raises whether these different applications have some parts in common. The most important ontological commitment behind the component-based approach is now that we may in fact view a single software component as used by many different applications. This is nothing else as to say that parts from distinct wholes need not to be distinct. Figure 1 tries to depict the ontological commitments that form the basis of the component-based approach.

We may now associate the cost of developing the applications in an organization with number of components that are shared. If we allow software components then cost of developing an application consists of the cost of developing the software components and of the cost of wiring the software

components to form applications. The cost of wiring is the additional cost of putting the parts together to make the whole. Assume that the overall cost of the applications in an organization is x , and that a particular software component costs c , and that this software component has been independently developed by $n \geq 0$ applications in the organization. If this software component were first developed and then shared by $n \geq k \geq 0$ applications, then the overall cost changes to $x(k) = x + c - k \cdot c$. Clearly if the software component is not used at all, there would be an additional cost. Only if the software component were used more than once, the cost would lower:

That a software component is not used at all is not an unusual case for an organization. For example a software component could have been developed for an application that turned out not to deliver the expected business benefit. In our case this is the profitable deployment (Moss 2000) of an agent-based computer model and simulation. So a part could have been created but the whole could have been dismissed. Or a software component itself can turn out not to fulfil the functional and/or non-functional requirements of an application. So a part could have been created but it could turn out that it does not serve the whole. Thus the development of a software component can be a risky investment. Only after a software component has been successfully applied and is applied again, additional payback results when the software component is shared.

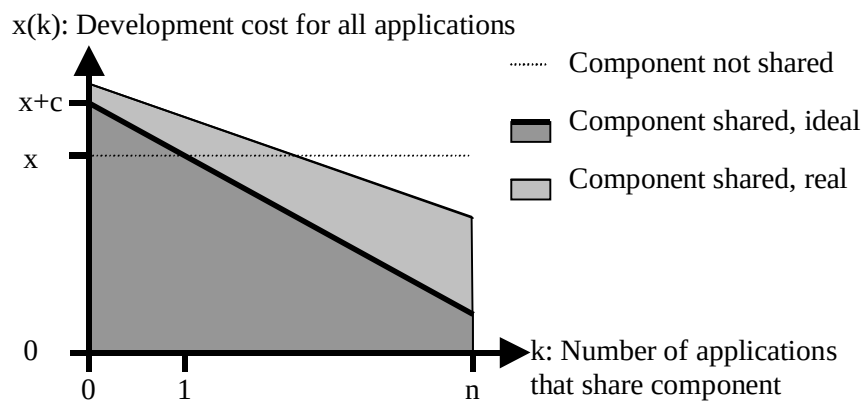


Figure 2: Benefit of first developing and then sharing a software component

It should be further noticed that business requirements could change. In our case this means that the conditions of application (Moss 2000) of an agent-based computer model and simulation could cease to hold. Thus a profitable deployment of an agent-based computer model and simulation can suddenly become impossible. New investment might then become necessary to accomplish a turn-around. This might include that applications are improved or that applications are replaced by other applications. External changes might ripple down to the conclusion that a single software component is not used anymore whereas other software components might survive. As a result certain software component will get less used over time and it might even become necessary to invest in the development of new software components

To get the additional payback of sharing the similar software components across applications have to be identified. This effort can result in additional costs that might in turn annihilate the payback that one gets from sharing. The overhead could be reduced to some degree by specification and repository techniques. But also the sharing itself might include some cost. Namely the underlying technology of object orientation does not come without a performance and storage penalty. Also if software components evolve then updates have to be distributed and requirements have to be synchronized to yield complete versions. Finally the wiring of software components can be more costly than writing applications from scratch. Therefore in figure 2 there is an ideal and real graph of the cost of sharing.

Agent-Based Simulation Components

Quicksilver provides software components and custom wiring in the area of agent-based computer models and simulations. It does so in complementing the Java platform. The modeler thus has to access at the same time the Java platform and Quicksilver for his modeling business. The cooperation with the Java platform is motivated by the additional costs that are involved in the component-based approach. Here the Java platform provides solutions. In particular the Java platform provides sharing and distribution of software components. The Quicksilver software components do inherit the sharing and distribution of the Java platform. The software components provided by Quicksilver are nothing else than Java software components. They complement the software components that are already part of the library that comes with the Java platform. The software components that Quicksilver delivers fall into the categories weak, strong and data agents.

The weak, strong and data agents software components are directly used as a basis for the model and analysis agents of the computer models and simulations that are developed with the help of Java and Quicksilver. The basic software components have been carefully designed to provide the recurrent core functionality of agent-based computer models and simulations in a flexible manner. The modeler should be able to use the software components without fear of running into restrictions that would not come in his way by developing his application from scratch. To identify the basic software components the modeler has first to divide his application into model and analysis agents. The model agents are supposed to represent the target domain. The analysis agents are used to calibrate, validate, etc. the target domain.

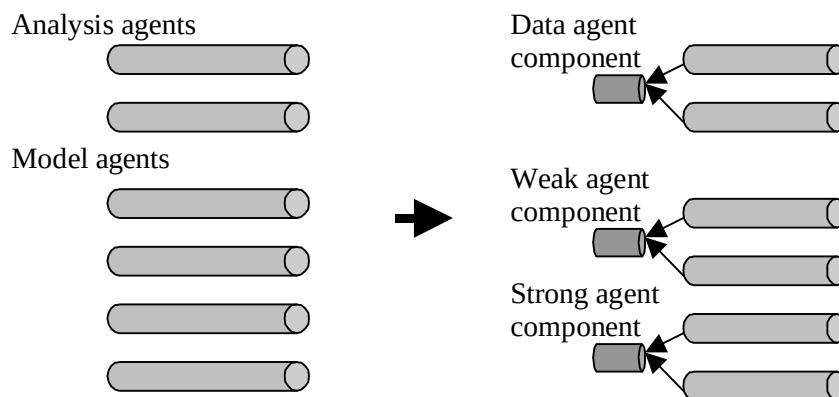


Figure 3: Software Components for the Generalization of Agents

To identify the basic software components every analysis and model agent has to be broken up into its core functionality and into a target domain dependent part. For the core functionality an appropriate basic software component from Quicksilver can then be chosen. The target domain dependent part is developed on top of the basic software component. The weak and strong agent components reflect the notions of a weak and strong agents as defined in (Woolridge 1994), whereas the data agents are supposed to provide scientific methods. The weak agents are simply active objects, whereas the strong agents are entities with representation and reasoning mechanisms. For example in a water management model the weak agents can be used to model the pumps and pipes of the infrastructure. The strong agents can then be used to model stakeholders such as engineers and consumers. Further a data agent could compute and display some mean/variance analysis of a measures on pumps and consumers.

Software components can be divided into classes and instances. Classes have different characteristics from instances in two respects. First it is assumed that classes are not great in number whereas instances can be great in number. Second classes can contain program code whereas instances can only contain data values. By means of the Java byte code, the Java platform defines a standardized format for classes. The basic tools of the Java platform are especially oriented towards the wiring of classes and thus the production of Java byte code. In an agent-based computer model and simulation classes are instrumented to represent expert knowledge. To be able to run simulations expert knowledge has to be in an operational form. This is archived by providing the expert knowledge as the program code of classes. Expert knowledge is found in the target domain dependent part and in the scientific methods.

Whereas classes are instrumented to represent expert knowledge the instances are instrumented to represent parameters and/or empirical data. When instances function as parameters they are connected to classes with expert knowledge. The data values in the instances are then used as parameters to the program code of these classes. There is a mechanism in object orientation that provides this dependency between instances and classes. By means of this mechanism even general expert knowledge can be capture in the form of software components that are classes The general expert knowledge might then be specialized by other classes or instances. When instances function as empirical data, they are simply part of the analysis and model agents. They might then be used to calibrate, validate, etc. the computer model.

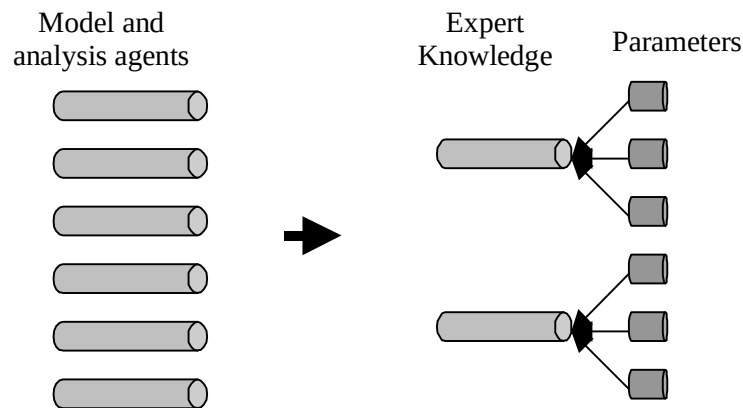


Figure 4: Software Components for the Specialization of Agents

Even to develop very simple agents the agents have to be broken up into expert knowledge and parameters because mode and analysis agents are always instances in the end. If one wants to take advantage of sharing the expert knowledge can be broken up further. There are currently no predefined target domain dependent general expert knowledge components in Quicksilver. These components have to be developed by the modeler on his own. Quicksilver only comes into play when instances have to be managed. Namely Quicksilver supports the wiring of instances. The editor of Quicksilver allows the interactive creation and composition of agent populations. The browser of Quicksilver allows the interactive testing of agent populations. In particular simulation runs can be performed during which data analysis software components can provide special views on aspects of the actual state and trace of the model.

Current Further Development

We have seen how Quicksilver is based on the component-based approach. Quicksilver delivers software components for weak agents, strong agents and data agents. These software agents are predefined generalizations especially designed for agent-based computer models and simulations. To

develop a model the modeler has to specialize these components and to invent his own generalizations. In the following a more detailed description of the predefined generalizations is given. The descriptions include potential further developments and the current uses:

Weak Agents: This category includes the classes "Agent" and "Swarm". The class "Agent" and tries to capture the notion of a weak agent as documented in (Woolridge 1994). The modeler can derive an active object in his model from this class. The current scheduling is such that an active object is called every simulation step. An active object is free to change the linkage of objects and/or to create objects, the simulation can thus use variable structures. The linkages between active objects may form any network and must not form a tree. The class "Swarm" is a refinement of the class "Agent", it tries to capture the notion of hierarchical structures (Troitzsch 1996). By means of the class "Swarm" the modeler can collect active objects in hierarchical activity clusters. There is still some development concerning the semantics of the "Swarm" class, namely its role as a context for the lifetime of agents and its role as a context for the activity of agents. Also it might become necessary to introduce more sophisticated scheduling which will affect the semantics of both the "Agent" class and the "Swarm" class. These classes are currently applied in (Tillman 2000, Hare 2000 and Kottotau 2000).

Strong Agents: This category includes the classes "Classic" and "Quantum". These classes cater for the notion of a strong agent as documented in (Woolridge 1994) and are supposed to form a basis for deliberative agents (Doran 1996) and uncertainty analysis (Varis 1994). The class "Classic" adds Prolog knowledge bases to the Java Platform such that one can handily use pattern matching, backtracking and procedural attachments. The class is an adaptation of (Burse 1994) and the implemented language and its semantics conforms to (ISO 1995). The class "Quantum" provides probabilistic inferencing (Spiegelhalter 1993), decision optimization and learning for belief networks. The implementation (Burse 2000) uses heuristics similar to (Cooper 1990). This class will probably undergo some extensions in the near future, as game theoretic reasoning and dynamic beliefs are still missing. The class "Quantum" is currently applied in (Hare 2000).

Data Agents: This category includes the classes "Viewer" and "MonteCarlo". The modeler can derive custom views for his agents from the class "Viewer". There are various visual components in stock such as graph component, network component, etc.. that can be used to build these viewers. The visual components have been applied in (Pahl 2000a, Schlumpf 2000, Pahl 2000b and Schönborn 1999). The viewers can be used to browse and even to edit agents. The agents have to provide the data for the viewers in that they monitor the state or trace of the model. The class "MonteCarlo" is an attempt of a first more sophisticated analysis agent. An instance of this class can be inserted into a simulation and it will perform a Monte Carlo analysis of the model. Various additional visual components and data agents can be conceived and will be developed if there is demand. The class "MonteCarlo" is currently applied in (Tillman 2000 and Hare 2000).

Quicksilver also delivers a tool for the wiring of instances of classes which will be the agents of a model. The tool supports the creation, composition and testing of instances. The issue of wiring of instances is not the only relevant issue with respect to the component-based approach. We give in the following a more complete list of the relevant issues that we are going to address in the Quicksilver environment:

Wiring Tool: The tool is built by means of contemporary techniques (Shneiderman 1991). It cooperates with the Java platform in that it makes use of the meta-information generated from the compilation of classes. Therefore the end-user can define a Java class with its own basic Java tools and then switch to the wiring tool. He will then immediately be able to edit and save instances of this new class. The wiring tool allows to re-socialize agents, namely one can copy a swarm from a model

and paste it to an other model. The wiring tool then re-establishes all internal links of the agents and will set up new external links if possible. To work with the agents the end-user has available the usual human computer interface universals as edit/browse, cut/copy/paste and new/open/save. The working of these universal is dependent on the viewer components that are provided for the agents and can thus be extended by the end-user. Further development will include support for universals like print, find/replace and drag-and-drop. Also there is some development underway to allow animated runs of simulations.

Exchange Formats: For sharing and distribution sets of agents are written out as so called model files. Because classes can evolve independently of model files these files can be out of sync. The current implementation of the wiring tool uses a file format and exchange facilities where model files that are out of sync can still be processed. The wiring tool will automatically correct anomalies by particular defaults and the end-user can intervene by directly editing the model file with a text editor. This is very primitive and we would like to have a better approach for the synchronization of model files. We will probably switch to the extensible markup language (XML) format that originally comes from professional document management and which is now used in e-commerce arena. XML is designed to describe and exchange structured data over the internet and could be directly used for weak agents. XML allows for extensibility and there are already extensions which are relevant to strong agents. Namely for the class "Classic" the business rule markup language (BRML) (Grosz 1999) and for the class "Quantum" the belief network interchange format (BNIF) (Microsoft 1998).

Specification Techniques: The Quicksilver environment does not support the identification of similar components in a particular automated way. Such a support might exist in the Java environment that Quicksilver is cooperating with. For example the Java environment might support a repository for the retrieval of classes. The Quicksilver environment does also not support a particular process for the development of a computer model. Again such a support might exist in the present Java environment by means of recipes, forms and tools. Unfortunately the presence of a repository and a process influences the production cost and the quality of a model. To improve the situation we manually started to deploy the unified modeling language (UML) as indicated in (Burse 1998). UML is very close to weak agents (Burse 1999) and therefore favors part of our broad notion of computer model. UML was naturally applied in (Tillman 2000, Hare 2000 and Kottonau 2000). With the introduction of strong agents we will need to refine and extend our methods. Most of the current requirements will probably be reflected in improved data agents.

Conclusions

The Quicksilver environment delivers software components for weak agents, strong agents and data agents. These software agents are predefined generalizations especially designed for agent-based computer models and simulations. To develop a model the modeler has to specialize these components and to invent his own generalizations. Quicksilver also delivers a tool for the wiring of instances of classes which will be the agents of a model. The tool supports the creation, composition and testing of instances. The particular software components and the instance wiring might indeed allow for a more profitable development of the broad notion of computer model we would like to adhere to.

The guiding principle behind the development of Quicksilver was the component-based approach. It is therefore often difficult to cast Quicksilver in more traditional categories from computer science. Take for example the category language. Except for the Java byte code no requirement is made on the components that might interact in the Quicksilver environment. It is thus open with what kind of tools and languages the classes were produced. The components are also allowed to have their own

viewers. The components might thus use a multitude of representations during the building and execution of a simulation. Now if the category language fails, then take for example the category grand model. It should be noted that the modeler is always free to exclusively use the weak agents. This component is very domain neutral and it could be used to model a door knob and a firm in the same time. Also the strong agents are still domain neutral as they only provides forms classical and probabilistic reasoning. Such reasoning is only a more elaborate layer of operation over the primitive imperative layer of Java code operation.

It should be noted that the development of Quicksilver needed some investment. It will now be of interest whether this investment will payback. What will be the next successful applications? In what new components will these new applications result? In which cases will the Quicksilver environment totally fail? Possible directions of further fundamental development are already indicated in research elsewhere. For example (Roschelle 1998) documents issues on project management and intellectual property. How is it accounted for when components from different vendors are quickly assembled into applications? (Chandy 1998) then reports on verification and validation. How can we guarantee properties of applications when components from different vendors with changing implementations are dynamically assembled together?

References

- Burse, J.
 [1994] JB-Prolog: Fast Prolog Demo System for the Macintosh, CMU Artificial Intelligence Repository, 1994
 [1999] A Basic Model of Parallelism, Semesterwork, Postgraduate Course on Intelligent Agents, EPFL Lausanne, Switzerland, November 1999
 [1998] Modeling Organized Complexity: A Case Study, OOPSLA'98 Workshop on Complex Adaptive Systems, Vancouver, October 19th
 [2000] Belief Networks, Mean/Variance Analysis and Bounded Rationality, SIAM Notes, EAWAG, in preparation
- Chandy, K. M., Sivilotti, P.A.G., Kini, J.R.
 [1998] A Cottage Industry for Software Publishing: Implications for Theories of Computation, Workshop on Formal Methods for Parallel Programming and Applications : Theory and Applications, Orlando, Florida, April 3, 1998
- Chattoe, E.
 [1996] Why Are We Simulating Anyway? Some Answers from Economics, in Social Science Microsimulation, Troitzsch, K.G. et al., Springer Verlag
- Cooper, G.F.
 [1990] Bayesian Belief-Network Inference Using Recursive Decomposition, Report KSL-90-05, Knowledge Systems Laboratory, Stanford University, California
- Doran, J.
 [1996] Simulating Societies using Distributed Artificial Intelligence, in Social Science Microsimulation, Troitzsch, K.G. et al., Springer Verlag
- Grosz, B.N. and Labrou, Y.
 [1999] An Approach to using XML and a Rule-based Content Language with an Agent Communication Language, IBM Research Report, RC 21491
- Hare, M.
 [2000] Uncertainty Analysis in an Agent-Based Integrated Nitrate Pollution model, submitted.
- Microsoft
 [1998] XML Belief Network File Format, Microsoft Decision Theory

ISO

- [1995] Programming Language Prolog Part I: General Core, ISO/IEC 13211-I, Working Draft, 3. August 1995

Kottonau, J.

- [2000] A Consumer Memory based Model of New Product Diffusion within a Social Network, submitted.

Luenberger, D.G.

- [1979] Introduction to Dynamic Systems: Theory, Models and Applications, John Wiley, New York.

Moss, S., Pahl-Wostl, C. and Downing, T.

- [2000] Agent based integrated assessment modeling, submitted

Pahl-Wostl, C. J.

- [2000a] The transition to a low-energy society: An evolutionary growth model, submitted.

Pahl-Wostl, C., Schlumpf, C., Schönborn, A., Büssenschütt, M. and Burse, J.

- [2000b] Models at the Interface between Science and Society: Impacts and Options, in press, Integrated Assessment.

Roschelle,

- [1998] Scaleable Integration of Educational Software: Exploring the Promise of Component Architectures, Journal of Interactive Media in Education, 98(6)

Schlumpf, C., Pahl-Wostl, C., Schönborn, A., Jaeger, C.C. and Imboden, D.M.

- [2000] IMPACTS: An Information Tool for Citizens to Assess Impacts of Climate Change from a Regional Perspective, submitted.

Schönborn, C., and Pahl-Wostl, C.

- [1999] The Personal Water Calculator, Proceedings of the 4th International Conference on Ecological Engineering for Wastewater Treatment, Agricultural University of Norway, Ås, Norway, June 7-11, 1999

Shneiderman, B.

- [1991] Designing the User Interface : Strategies for Effective Human-Computer Interaction, 3rd Edition, Addison-Wesley

Spiegelhalter, D.J., Dawid, A.P. , Lauritzen, S.L. and Cowell, R.G.

- [1993] Bayesian Analysis in Expert Systems, in Statistical Science, 8(3), 219-283

Troitzsch, K.G.

- [1996] Multilevel Simulation, in Social Science Microsimulation, Troitzsch, K.G. et al., Springer Verlag

Tillman, D., Larsen, T., Pahl-Wostl, C. and Gujer, W.,

- [2000] Interaction Analysis of the Stakeholders in Water Supply Systems, in press.
- [1999] Modeling the Actors in Water Supply Systems, Water Science and Technology 39: 203-211, 1999.

Varis, O.

- [1994] A Belief Network Approach to Modeling of Environmental Change: The Methodology and Prospects for Application, WP-94-04, May 1994, IIASA, Laxenburg, Austria

Wooldridge, M. and Jennings, N.R.

- [1994] Agent Theories, Architectures and Languages: A Survey, in Intelligent Agents, ECAI-94 Workshop, Amsterdam, The Netherlands, August 1994, LNAI 890